

Forecasting Methods: Calendar, Reactive, and Mix

CONTENTS

1. [Overview](#)
2. [1\) Calendar Forecast](#)
3. [2\) Reactive Forecast](#)
4. [3\) Mix Forecast \(Weekly Weights\)](#)
5. [4\) Worked Example](#)
6. [5\) Evaluation & Fine-Tuning](#)
7. [6\) Gradient Descent: Math, Tiny Example, and Code Sketch](#)

Overview

This note consolidates three complementary forecasting approaches—*Calendar*, *Reactive*, and a *Mix* of both—followed by evaluation metrics and a light optimization sketch. The core idea is practical: exploit stable calendar structure, add a short-term reactive signal, and blend them with simple weekly weights. We keep loss functions and KPIs aligned with business risk, with a slight bias against overestimation or underestimation depending on context.

1) Calendar Forecast

The calendar component uses *damped Holt* smoothing for level and trend, plus multiplicative calendar/event effects smoothed by exponential averages.

Level update:

$$\hat{y}_t = \alpha y_t + (1 - \alpha) (\hat{y}_{t-1} + \phi b_{t-1})$$

Trend update:

$$b_t = \beta (\hat{y}_t - \hat{y}_{t-1}) + (1 - \beta) b_{t-1}$$

Effects are defined from historical multiplicative deviations $e_t = \frac{y_t}{\hat{y}_t}$, grouped by calendar buckets and smoothed. Future forecasts combine the level line s_τ with the product of bucket multipliers:

$$\hat{y}_\tau^{\text{cal}} = s_\tau \cdot \prod_{f \in \mathcal{F}} \tilde{e}_{k_f(\tau)}$$

2) Reactive Forecast

Captures short-term uplift vs. last year using a 7-day window and a 364-day lag to preserve weekday alignment.

Increase rate:

$$\rho = \frac{1}{7} \sum_{i=0}^6 \frac{y_{T-6+i}}{y_{T-6+i-364}}$$

Forecast rule:

$$\hat{y}_{T+h}^{\text{react}} = \rho \cdot y_{T+h-364}$$

3) Mix Forecast (Weekly Weights)

We blend calendar and reactive components using weights that depend on the forecast week $w(h)$ (Week 1, Week 2, ...). Let c_w and r_w be non-negative and sum to one:

$$\hat{y}_{T+h} = c_{w(h)} \hat{y}_{T+h}^{\text{cal}} + r_{w(h)} \hat{y}_{T+h}^{\text{react}}, \quad c_w + r_w = 1.$$

Weights can be learned on historical rolling windows or set heuristically (e.g., more reactive in Week 1, more calendar later).

4) Worked Example

Suppose $\rho = 1.05$. The table below illustrates, for the first four forecast weeks, how the Calendar and Reactive components combine with weekly weights to produce the final forecast.

Week (DOW)	c_w	r_w	$\hat{y}^{\text{cal}}$	$\hat{y}^{\text{react}}$	Final $\hat{y}$
1 (D0)	0.6	0.4	112.0	106.0	$0.6 \times 112 + 0.4 \times 106 = 109.6$
2 (D3)	0.65	0.35	118.0	108.0	$0.65 \times 118 + 0.35 \times 108 = 114.5$
3 (D1)	0.7	0.3	115.0	105.0	$0.7 \times 115 + 0.3 \times 105 = 112.0$
4 (D6)	0.75	0.25	120.0	110.0	$0.75 \times 120 + 0.25 \times 110 = 117.5$

Interpretation: Early weeks lean more on the reactive signal; later weeks preferentially trust the calendar baseline.

5) Evaluation & Fine-Tuning

Primary Loss

We use a smoothed asymmetric percentage loss to encode business risk:

$$\text{AMAPE}_t = \frac{|y_t - \hat{y}_t|}{\max(\epsilon, |y_t|)}, \quad w_t = \begin{cases} \lambda, & \hat{y}_t < y_t \\ 1, & \text{otherwise} \end{cases}, \quad \mathcal{L} = \frac{1}{T} \sum_{t=1}^T w_t \text{AMAPE}_t.$$

Choose $\lambda > 1$ (e.g., $\lambda = 2$) to reflect shortage risk; ϵ prevents division by zero.

Secondary KPIs

- 1% tolerance band: $\Pr(\hat{y}_t \in [0.99 y_t, y_t])$
- Directional accuracy vs. week-ago change
- P95 absolute percentage error on peak weeks

Offline Training Loop

```

for each rolling origin:
    fit calendar / reactive components
    learn weekly weights {c_w, r_w}
  
```

score on holdout with primary loss + KPIs
aggregate results; choose λ , horizons, and weights

6) Gradient Descent: Math, Tiny Example, and Code Sketch

Parameters $\theta = (\alpha, \beta, \phi)$ are box-constrained to $[0, 1]^3$. We perform projected gradient steps on the weighted loss:

$$\theta \leftarrow \Pi_{[0,1]^3}(\theta - \eta \nabla_{\theta} \mathcal{L}).$$

For numerical stability in the derivative of AMAPE, we use $\sqrt{(y_t - \hat{y}_t)^2 + \delta^2}$ in the denominator:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_t} \approx -\frac{\text{sign}(y_t - \hat{y}_t)}{\max(\varepsilon, |y_t|)} \cdot \frac{|y_t - \hat{y}_t|}{\sqrt{(y_t - \hat{y}_t)^2 + \delta^2}}.$$

Tiny Numerical Example

Given $y_t = 110$, $\hat{y}_{t-1} = 100$, $b_{t-1} = 0$, $\phi = 1$, $\alpha = 0.2$, $\lambda = 2$, $\varepsilon = 10^{-6}$, $\delta = 10^{-3}$:

$$\begin{aligned}\hat{y}_t &= 0.2 \cdot 110 + 0.8 \cdot 100 = 102, & e_t &= y_t - \hat{y}_t = 8, & (w_t &= 1). \\ \frac{\partial \mathcal{L}}{\partial \hat{y}_t} &\approx -\frac{1}{110} \cdot \frac{8}{\sqrt{8^2 + 10^{-6}}} \approx -0.07273, & \frac{\partial \hat{y}_t}{\partial \alpha} &= 110 - 100 = 10, \\ \frac{\partial \mathcal{L}}{\partial \alpha} &\approx (-0.07273) \times 10 = -0.7273.\end{aligned}$$

With learning rate $\eta = 0.05$ and initial $\alpha_0 = 0.2$, a single step is:

$$\alpha_1 = \Pi_{[0,1]}(\alpha_0 - \eta \frac{\partial \mathcal{L}}{\partial \alpha}) = \Pi_{[0,1]}(0.2 - 0.05 \times (-0.7273)) = \Pi_{[0,1]}(0.2364) = 0.2364.$$

Projection is inactive since $0.2364 \in [0, 1]$.

Code Sketch (pseudo)

```
# Pseudocode: optimize theta = (alpha, beta, phi) on validation
theta = dict(alpha=0.2, beta=0.5, phi=0.9) # init
eta = 0.05
lam = 2.0
eps = 1e-6
delta = 1e-3

for epoch in range(E):
    L_epoch = 0.0
    # roll one-step-ahead forecasts across validation window
    level, trend = init_from_train(...) # carry state
    for t in val_index: # chronological
        y = actual[t]
        yhat = theta['alpha']*y + (1-theta['alpha'])*(level + theta['phi']*trend)

        # asymmetric smooth percentage loss
        w = 1.0 if yhat <= y else lam
        err = y - yhat
        L = w * ( (err*err + delta*delta)**0.5 ) / (y + eps)
        L_epoch += L

    # gradients for alpha only (extend similarly for beta, phi)
    dL_dyhat = -(w/(y+eps)) * (err) / ((err*err + delta*delta)**0.5)
    dyhat_dalpha = y - (level + theta['phi']*trend)
    dL_dalpha = dL_dyhat * dyhat_dalpha
```

```
# gradient step with projection
theta['alpha'] = min(1.0, max(0.0, theta['alpha'] - eta * dL_dalpha))

# update level/trend with current theta (stateful)
new_level = theta['alpha']*y + (1-theta['alpha'])*(level + theta['phi']*trend)
new_trend = theta['beta']*(new_level - level) + (1-theta['beta'])*trend
level, trend = new_level, rolling_mean_trend_update(trend, new_trend)
```